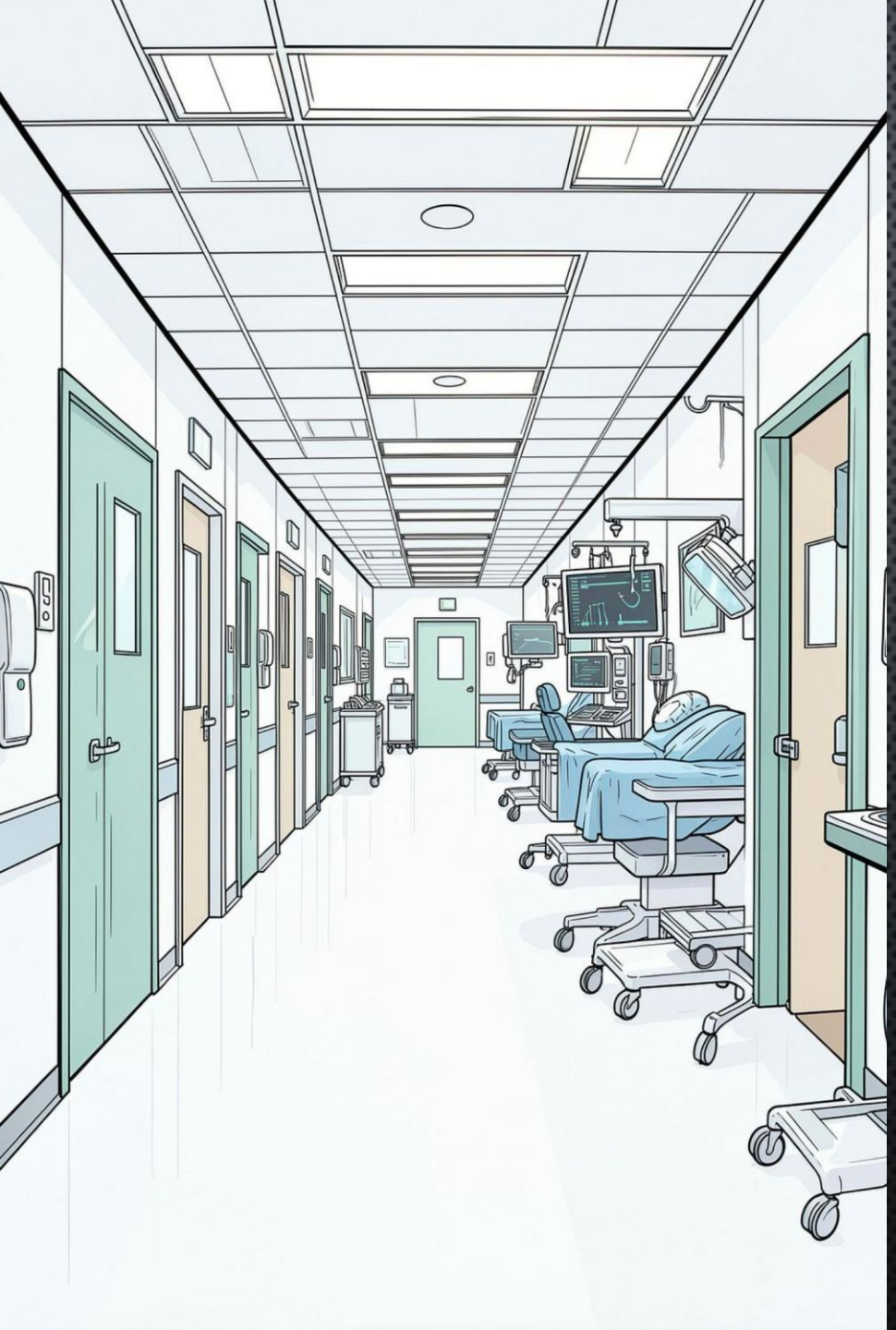
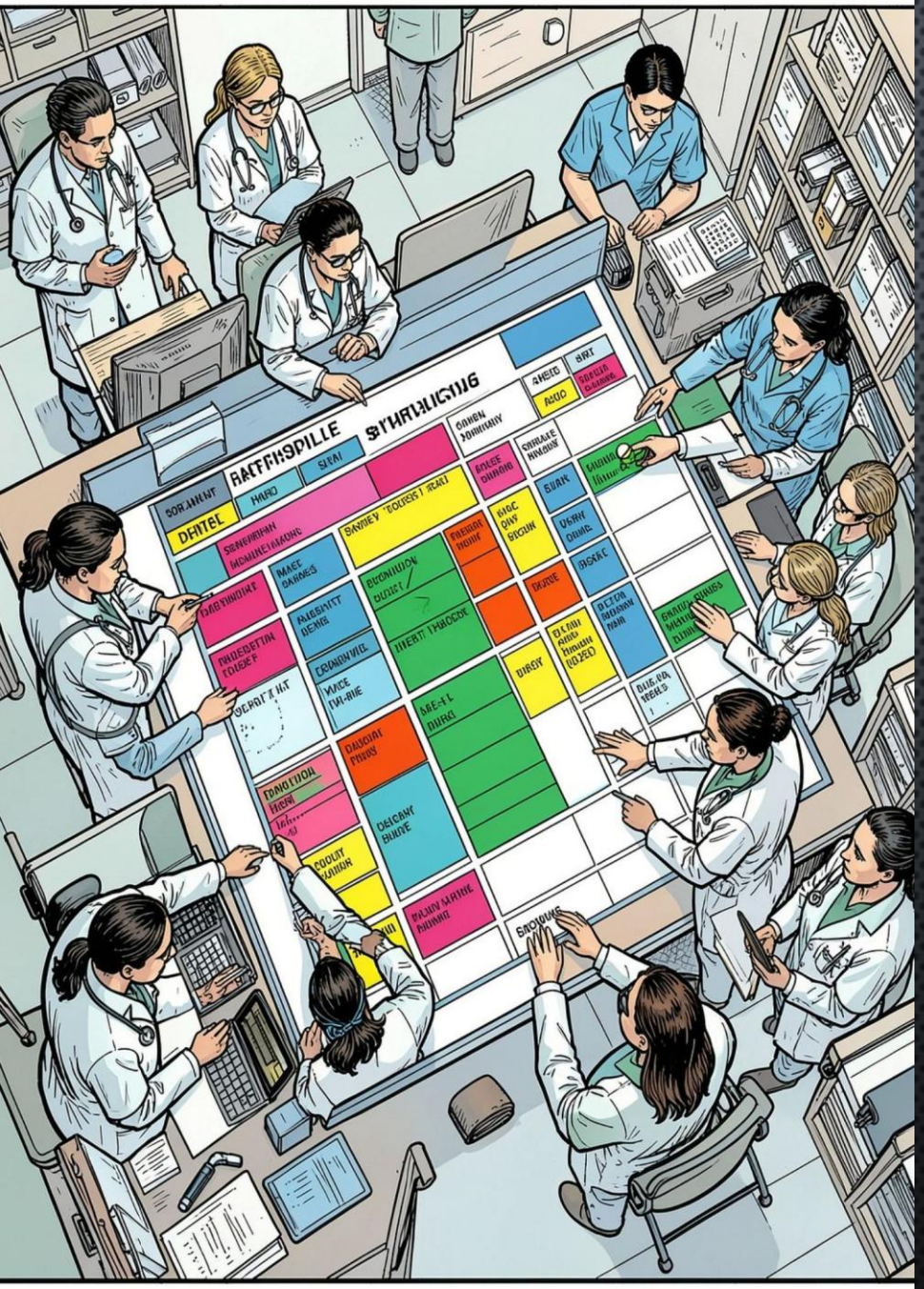


Hospital Bed & OR Scheduling

An intelligent optimization engine for dynamic allocation of patients, beds, and operating rooms — minimizing idle time, maximizing throughput, and improving clinical outcomes.

OPERATIONS RESEARCH · HEALTHCARE IT

M.Shiva - 2520090135
P.Bhavan-2520090146



The Scheduling Challenge

Hospitals must simultaneously balance procedure durations, patient priorities, staff availability, and strict deadlines — a problem that grows exponentially complex at scale.

⏱️ Idle Resources

Unfilled OR slots and vacant beds cost hospitals thousands per hour in lost capacity.

🚨 Longer Waits

Poor scheduling cascades into delayed procedures, patient dissatisfaction, and compromised outcomes.

⚡ Dynamic Disruptions

Emergencies, cancellations, and staff absences require real-time schedule adaptation.

Project Abstract



This project develops an **optimization engine** that intelligently allocates patients and procedures to rooms and time slots. The system minimizes makespan and idle time while strictly respecting all operational constraints.

Resource Utilization

Algorithmic assignment of beds, ORs, and staff to maximize throughput.

Priority & Deadline Handling

Urgency-weighted scheduling ensures critical patients are prioritized.

Reporting & Transparency

Built-in analytics on utilization rates, bottlenecks, and quality metrics.

Introduction: Why This Matters

Operating rooms account for **up to 40% of hospital revenue** and a significant portion of total costs. Even marginal improvements in scheduling efficiency translate to major gains in patient care and financial performance.

40%

Revenue Share

ORs represent the largest revenue-generating unit in most hospitals.

30%

Avoidable Idle Time

Estimated OR downtime attributable to suboptimal manual scheduling.

24/7

Operational Demand

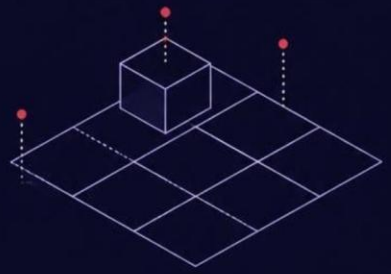
Hospitals require round-the-clock scheduling resilience and adaptability.

Literature Survey

Research in healthcare scheduling spans operations research, constraint programming, and machine learning — each offering distinct strengths for this domain.

1) Integer Programming

exact optimization for small-to-medium problems



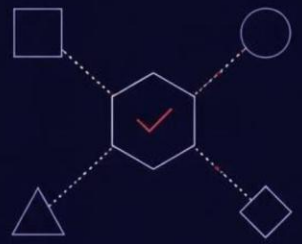
2) Heuristic Algorithms

genetic algorithms and simulated annealing for near-optimal solutions at scale



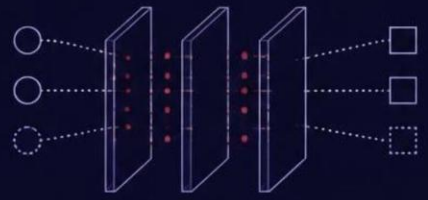
3) Constraint Satisfaction

rule-based systems enforcing hard and soft constraints



4) Machine Learning

predictive models for procedure duration and patient flow



Key Findings

- Meta-heuristic approaches outperform exact methods on large, dynamic instances
- Real-time rescheduling capability is critical for emergency responsiveness
- Hybrid models combining rule-based logic with optimization yield best practical results
- Patient priority weighting significantly improves clinical outcome metrics

Methodology

1

Problem Formulation

Define constraints: bed types, OR availability, staff shifts, patient acuity, and procedure durations.

2

Algorithm Design

Select and tune optimization approach — greedy initialization with local search refinement.

3

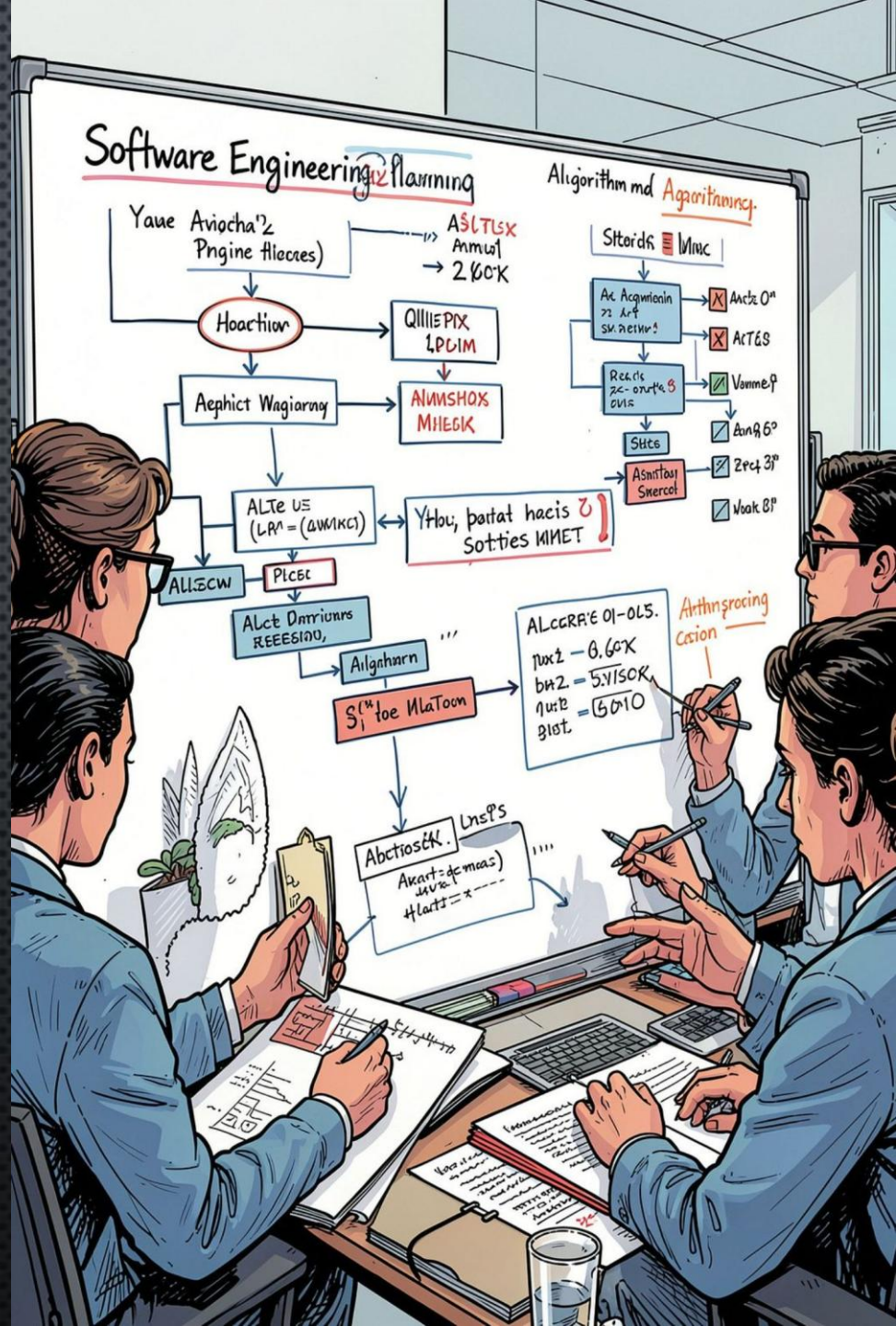
Integration Layer

Connect engine to hospital data systems via APIs for real-time input and schedule output.

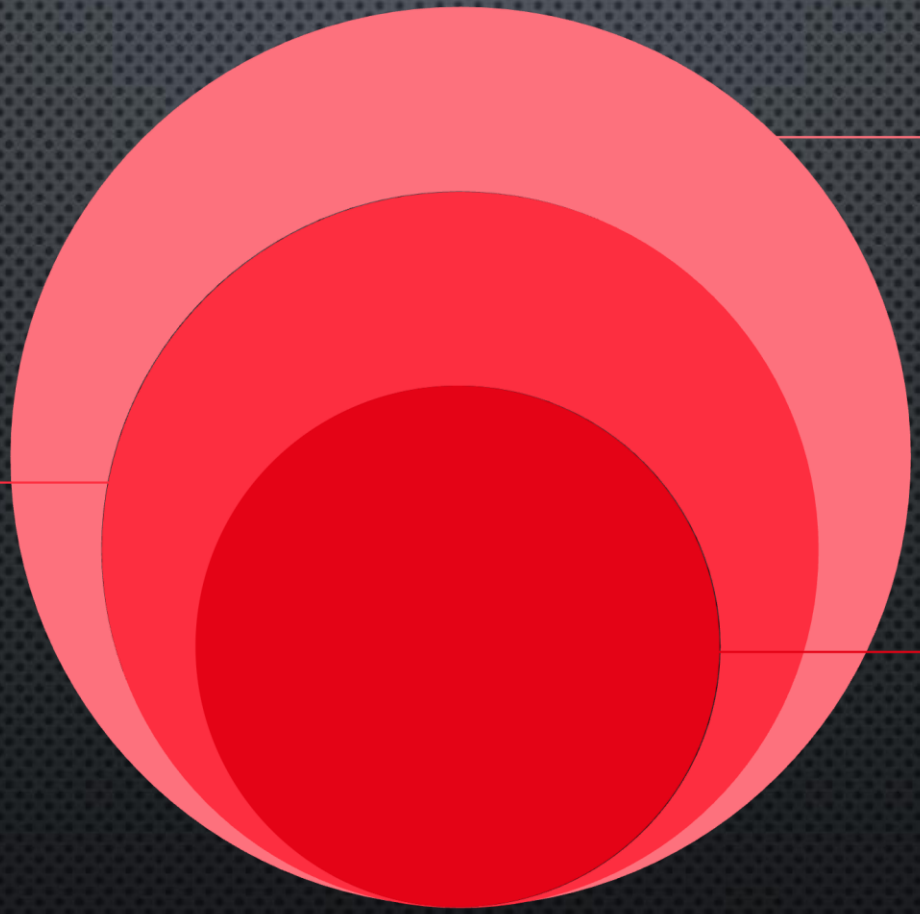
4

Validation & Reporting

Test against historical data; generate utilization dashboards and bottleneck reports.



System Architecture



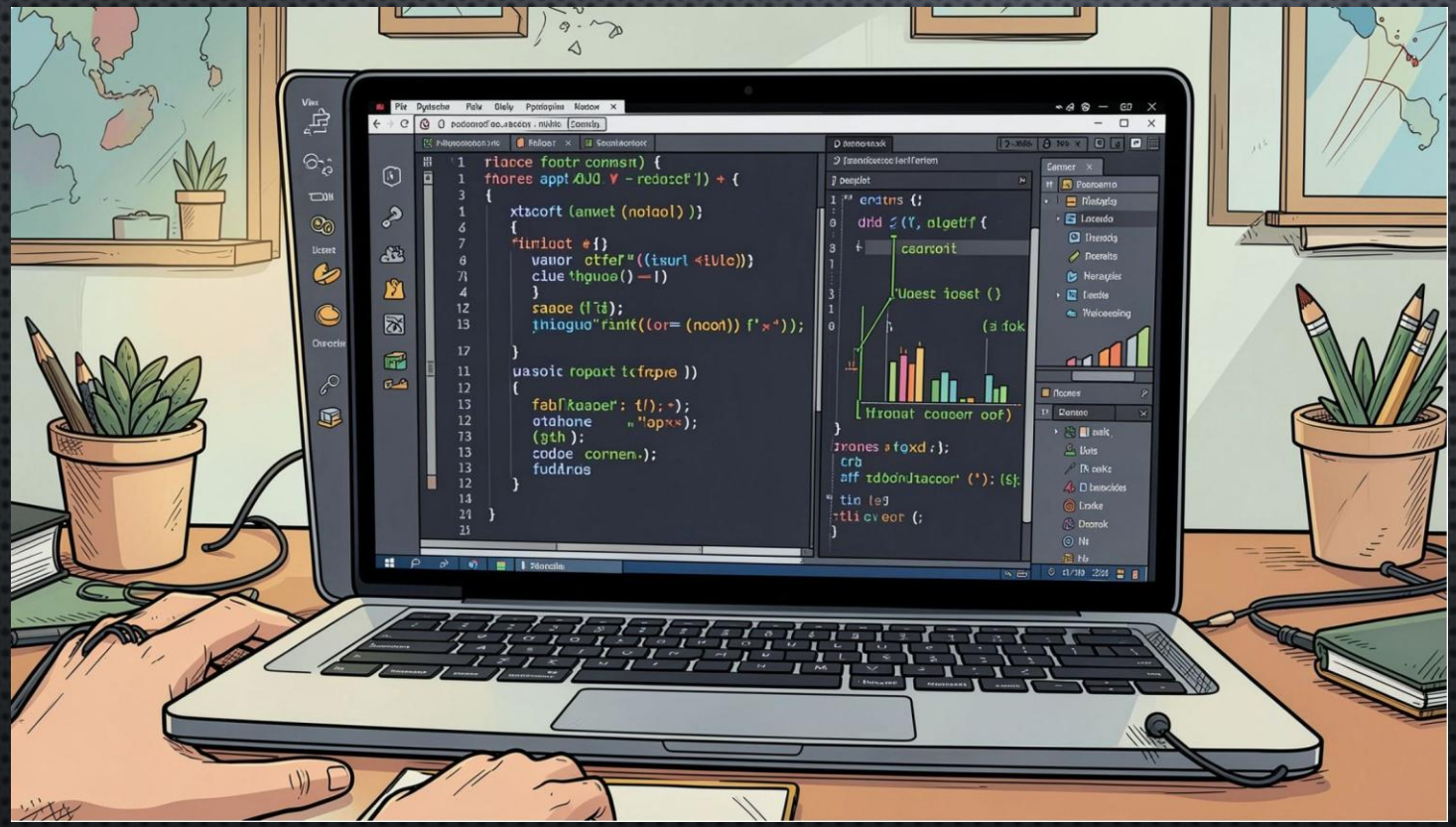
Output & Reporting
Schedules, allocations, dashboards and alerts

Input Layer
Patient data, staff rosters, room availability

Core Engine
Optimization engine with solvers and scheduler

The engine operates as a modular pipeline — each layer can be updated or replaced independently, enabling integration with existing hospital information systems (HIS) and electronic health records (EHR).

Software Requirements



Core Stack

Backend

Python 3.10+ with optimization libraries (PuLP, OR-Tools, or Gurobi)

Database

PostgreSQL for structured scheduling data; Redis for real-time caching

API Layer

RESTful API (FastAPI or Flask) for integration with hospital systems

Frontend / Reporting

React dashboard with Chart.js or D3.js for utilization visualizations

Hardware Requirements



Application Server

Minimum 8-core CPU, 32 GB RAM for optimization engine workloads. Recommended: 16-core, 64 GB for high-volume hospitals.



Database Server

Dedicated PostgreSQL instance on SSD-backed storage. 500 GB minimum; scalable via cloud infrastructure (AWS RDS, Azure SQL).



Network & Integration

Secure VPN or dedicated link to hospital HIS/EHR systems. TLS 1.3 encryption required for all data in transit.



Client Workstations

Standard clinical workstations with modern browsers (Chrome, Edge) for scheduler access to the reporting dashboard.

Expected Outcomes & Impact

By combining operational efficiency with patient-centric scheduling, this solution directly enhances hospital capacity management, staff productivity, and clinical outcomes.

📉 Reduced Makespan

Optimized procedure sequencing compresses total schedule length and eliminates gaps.

📈 Higher Utilization

OR and bed utilization rates improve through intelligent, constraint-aware assignment.

🔁 Dynamic Resilience

Automatic rescheduling responds to emergencies, cancellations, and staff changes in real time.

📊 Transparent Reporting

Live dashboards surface bottlenecks, quality metrics, and utilization trends for decision-makers.

